

DESCRIPCIÓN DEL PRODUCTO Y ARQUITECTURA DE SEGURIDAD

ODRE PQC v0.2.9

Application-Embedded Post-Quantum Security Boundary

Integración · Límite de seguridad · Operaciones

EDICIÓN EN ESPAÑOL

Septiembre de 2026

Mapa del documento

01	1. Descripción del Producto
02	2. Experiencia de Integración Mínima
03	3. Arquitectura del Producto
04	4. Contrato de Seguridad Fail-Closed
05	5. Platform Adapter y Native Runtime
06	6. Diagnóstico Operativo
07	7. Arquitectura de Worker y Lifecycle
08	8. Límites de Confianza y Superficie de Ataque
09	9. Arquitectura de Despliegue
10	10. Identidad del Producto
11	11. Conclusión

Descripción del Producto y Arquitectura de Seguridad

Application-Embedded Post-Quantum Security Boundary

ODRE PQC es un producto de seguridad multiplataforma que instala un límite de protección verificable delante de rutas seleccionadas de una aplicación. El desarrollador no tiene que ensamblar los detalles de la criptografía poscuántica. La integración pública permanece mínima, mientras el contrato interno combina confianza en el Native Runtime, Handshake criptográfico, Session, Strict Sequence, defensa Replay, semántica de solicitudes, respuestas protegidas, control de Lifecycle y comportamiento Fail-Closed.

1. Descripción del Producto

1.1 Definición

ODRE PQC no es una biblioteca que solo expone funciones PQC. Es una Application Security Boundary entre rutas FastAPI protegidas y un Native PQC Runtime. Antes de que una solicitud alcance el Business Handler, el producto verifica Runtime Identity, estado del protocolo, Session, continuidad de Sequence y significado autenticado de la solicitud.

El producto cubre:

- Identidad del Runtime y de los Artifacts instalados
- ML-KEM-768 Key Encapsulation
- ML-DSA-65 Digital Signature
- Hybrid Handshake y establecimiento de Session
- Expiración y eliminación de Session
- Strict Sequence y bloqueo Replay
- Binding semántico de Method, Path, Query, Header y Body
- Binding de Response y metadatos protegidos
- Terminación Fail-Closed antes del Handler
- Lifecycle de Worker, Crash, Restart y Shutdown
- Diferencias de Native Trust entre Windows y Ubuntu
- Diagnóstico operativo mediante [doctor](#), [verify](#) y [status](#)

1.2 Problema que Resuelve

Poder invocar un algoritmo PQC no equivale a procesar con seguridad una solicitud real. Un producto desplegable debe determinar qué Runtime se cargó, si el archivo verificado es el realmente cargado, cómo se vincula el Handshake con la Session, si un Replay puede ejecutar dos veces el Handler, si los metadatos HTTP externos alteran el significado y si un estado obsoleto sobrevive a un Crash.

ODRE PQC convierte estas preguntas en condiciones obligatorias antes de ejecutar un Handler protegido.

1.3 Principios Centrales

- Integración simple: el desarrollador no implementa la State Machine criptográfica.
- Significado vinculado de extremo a extremo: se protege lo que la aplicación procesa, no solo un Body cifrado.
- Sin ejecución cuando no hay prueba: las rutas protegidas no permiten bypass silencioso ni Automatic Classical-only Downgrade.

1.4 Entornos Objetivo

El producto se dirige a servicios FastAPI, APIs que requieren confidencialidad, integridad y defensa Replay, flotas mixtas Windows/Ubuntu y operaciones que necesitan diagnóstico por línea de comandos.

Elemento	Combinación verificada
Windows	Windows Server 2022 AMD64 / CPython 3.12.10
Linux	Ubuntu 24.04.4 LTS AMD64 / CPython 3.12.3
Runtime criptográfico	OpenSSL 3.5.8
Key Encapsulation	ML-KEM-768
Firma digital	ML-DSA-65
Wire Protocol	odre-pqc/3

1.5 Relación con TLS, VPN, WAF y Librerías PQC

Componente	Función principal	Diferencia con ODRE PQC
TLS	Confidencialidad e integridad del canal	ODRE PQC vincula Route, Session, Sequence y significado.
VPN	Limita el alcance de red	ODRE PQC verifica solicitudes incluso dentro de una red permitida.
WAF	Filtra patrones y políticas HTTP	ODRE PQC decide si una solicitud criptográficamente válida ejecuta el Handler.
Librería PQC	Proporciona primitivas	ODRE PQC compone primitivas en un Lifecycle Fail-Closed.
ODRE PQC	Límite embebido de aplicación	Un contrato desde Runtime Trust hasta Response Binding.

Son capas complementarias. ODRE PQC añade condiciones de ejecución de la aplicación por encima del transporte, la red y el filtrado HTTP.

1.6 Rutas Públicas y Protegidas

Las rutas públicas pueden mantenerse disponibles según la política de la aplicación. Las rutas protegidas solo se ejecutan después de verificar Runtime, Handshake, Session, Sequence y semántica. Si el Runtime no puede demostrar un estado seguro, las rutas públicas de información o Health pueden continuar y las protegidas se cierran: Fail-Closed with Public Availability.

2. Experiencia de Integración Mínima

2.1 Flujo de Instalación

```
Instalar Wheel
→ odre-pqc init
→ odre-pqc doctor
→ odre-pqc verify
→ install(app)
→ Iniciar aplicación
→ odre-pqc status
```

Etapas	Propósito
Instalación	Coloca el Package y los Entry Points de CLI.
init	Inicializa Runtime de plataforma y estado local.
doctor	Diagnostica entorno, permisos, archivos y requisitos del Runtime.
verify	Ejecuta Self-tests PQC reales y verifica la instalación.
install(app)	Registra la Security Boundary en FastAPI.
status	Lee el estado sin modificarlo.

2.2 Punto de Integración FastAPI

```
from fastapi import FastAPI
from odre_pqc import install

app = FastAPI()
install(app)
```

La superficie mínima separa responsabilidades. Los Handlers no administran ML-KEM Ciphertext, firmas ML-DSA, Replay Windows, Provider Paths ni Native Handles. La Boundary aplica el mismo orden de seguridad a cada solicitud protegida.

2.3 Responsabilidades de la Aplicación

La aplicación selecciona rutas protegidas, aplica autorización de usuarios y organizaciones, valida Business Input, define retención y decide qué funciones pueden permanecer públicas. ODRE PQC garantiza que la solicitud protegida cumpla el Security Contract antes de ejecutar esas políticas.

3. Arquitectura del Producto

3.1 Modelo por Capas

FastAPI Application

- ODRE PQC Integration Boundary
- Admission and Semantic Binding
- Handshake and Session Engine
- Sequence and Replay Gate
- Protected Handler

Integration Boundary → Platform Adapter → Native PQC Runtime

Las capas son Application Integration, Admission, Protocol, Semantic Binding, Platform Adapter y Native Runtime. Las diferencias del sistema operativo quedan debajo del contrato Python común.

3.2 Flujo de Solicitud Protegida

Solicitud protegida

- Admission
- Handshake o Session válida
- Strict next-sequence
- Request Semantic Binding
- Protected Handler exactamente una vez
- Response Binding y protección

El orden importa. Streaming no admitido, contratos de salida sin límite, Envelopes Malformed y entradas Oversized deben rechazarse antes de causar efectos secundarios del Handler.

3.3 Admission

Admission comprueba Message Type, campos obligatorios, Encoding, tamaño total, campos contradictorios, formas de Response admitidas y Security Metadata necesaria. Superar Admission solo significa que la siguiente capa puede interpretar la solicitud con seguridad; no significa que esté autenticada.

3.4 Handshake y Session

Client Material

- Runtime Identity y Self-test
- ML-KEM-768 / ML-DSA-65
- Bind de Transcript, Path y Version
- Server Material y Signature
- Key Confirmation
- Crear Session, Sequence = 0

Protocol Version, Algorithm Set, materiales de ambos extremos, Transcript, significado de la ruta y Session Identifier quedan vinculados. Si no se verifica una relación requerida, no se crea la Session.

3.5 Funciones Criptográficas

- ML-KEM-768: encapsula y recupera material secreto compartido.
- ML-DSA-65: autentica el material del Handshake y la integridad del Transcript.
- Hybrid Handshake: vincula los componentes requeridos al mismo Peer, Session y Route Meaning.

El éxito de una primitiva no establece protección por sí solo. También deben completarse Runtime Identity, Transcript Binding, Key Confirmation y Session. El fallo de un componente obligatorio no se convierte en éxito Classical-only.

3.6 Lifecycle de Session

Uninitialized → Handshaking → Active → Expired → Closed
↘ Validation Failure ↗

Una Session Active no es confiable indefinidamente. Expiry, cambio de Runtime, cambio de Worker Ownership, Sequence Violation o Integrity Failure cierran el estado protegido.

3.7 Strict Sequence y Defensa Replay

Cada solicitud protegida debe llevar exactamente el siguiente Sequence. Duplicate, Rollback y Gap se rechazan; en una carrera con valores idénticos solo se acepta uno. La decisión y la actualización son atómicas respecto al Handler Dispatch, evitando una doble ejecución.

3.8 Request Semantic Binding

Cifrar solo el Body deja Method, Path, Query y Headers significativos abiertos a sustitución. ODRE PQC vincula HTTP Method, Path y Query normalizados, Headers relevantes, Payload o Envelope, Session ID, Sequence, Protocol Version y Message Type. Una diferencia entre el significado autenticado y el visible para el Handler impide la ejecución.

3.9 Response Binding

Status Code, Content Type, Headers protegidos, Cookies, Response Body y la Session/Sequence correspondiente se vinculan al resultado. Las formas de respuesta que no puedan representarse con seguridad se rechazan explícitamente.

4. Contrato de Seguridad Fail-Closed

Fail-Closed significa que una solicitud cuyas condiciones de protección no pueden demostrarse no entra al Protected Handler. No exige detener todas las rutas públicas ni todo el proceso.

Las rutas protegidas se bloquean cuando no pueden demostrarse Runtime Identity, confianza de Provider o Dependency, Self-test PQC, Handshake Transcript, Session, Strict Sequence, Semantic Binding, Worker Ownership o Recovery seguro. Automatic Classical Downgrade no se considera protección equivalente.

Rechazar una respuesta después de ejecutar efectos de base de datos o servicios externos es demasiado tarde. Admission y las decisiones de contrato se colocan antes del Handler siempre que sea posible.

5. Platform Adapter y Native Runtime

5.1 Límite Multiplataforma

```
Common Python Security Contract
├─ Windows: DLL, ACL, Junction, Reparse, Loaded Module
├─ Ubuntu: SO, Permission, Symlink, FD, Inode, Loader
↓
Verified Native Runtime
```

5.2 Windows Native Trust

Windows cubre rutas DLL/Provider, ACL y Ownership, Junction/Reparse Bypass, Dependency Search Paths, Verified-versus-loaded Identity, sustitución del Runtime y vida de Handles/Modules. Un Hash correcto antes de la carga no basta si el Loader puede seleccionar otro objeto o Dependency.

5.3 Ubuntu Native Trust

Ubuntu cubre Shared Object y Parent Permissions, Symlink y Unsafe Parent, identidad (`st_dev`, `st_ino`), Atomic Replacement entre verificación y carga, inyección de Provider/Module/Engine/Config, Dependency Resolution y recuperación tras SIGKILL o Restart. FD e Inode vinculan transiciones críticas a objetos reales, no a rutas mutables.

5.4 Runtime Archive y Manifest

Se verifican selección de plataforma, Archive Hash, Hashes por archivo, permisos y Parent Trust, Loaded-object Identity y compatibilidad Package/Runtime. La inicialización pasa por Staging, bloqueo de Object Identity, Atomic Promotion, Native Load, Version Check y Self-tests reales ML-KEM/ML-DSA. La ausencia de primitivas o Identity Proof causa Fail-Closed.

6. Diagnóstico Operativo

- `init`: prepara Runtime y estado local, detectando inicialización concurrente, parcial o no confiable.
- `doctor`: diagnostica OS, Architecture, Python, Runtime Identity, permisos, Hashes, Symbols, Provider, Dependencies y State Store.
- `verify`: comprueba versiones, Native Load, Self-tests reales ML-KEM-768/ML-DSA-65, Hybrid Handshake, Session Lifecycle, Protected Round Trip y ausencia de Automatic Downgrade.
- `status`: informa el estado en modo Read-only sin modificar Sessions/Leases ni exponer Secrets.

7. Arquitectura de Worker y Lifecycle

Los Workers competidores no deben compartir silenciosamente Runtime o estado bajo un modelo no admitido. El Shutdown normal detiene nuevas admisiones protegidas, drena trabajo en curso con límite, elimina Sessions y Leases, cierra Native Resources y libera Locks en ese orden.

Crash, SIGKILL y pérdida de energía no pueden depender del Cleanup. El Restart verifica Process Ownership, Stale Locks, Sessions, Worker Leases, Status State y Runtime parcialmente inicializado.

Starting → Ready → Draining → Stopped
↓ ↓
Blocked Crashed → Recovering → Ready o Blocked

8. Límites de Confianza y Superficie de Ataque

Los límites principales están entre Untrusted Client y HTTP Parser, HTTP Metadata y semántica autenticada, ODRE PQC y Protected Handler, Python y Native Loader, Path String y Filesystem Object, y Worker State y Persistent State.

Superficie	Ataque representativo	Defensa
Admission	Malformed, Oversized, Smuggling Shape	Admission Gate
Session	Expired o Cross-session Reuse	Session Engine
Sequence	Duplicate, Rollback, Gap, Race	Atomic Sequence Gate
Request Meaning	Sustitución Method, Path, Query, Header	Semantic Binding
Response Meaning	Sustitución Status, Header, Cookie	Response Binding
Native Files	TOCTOU y Atomic Replacement	Platform Adapter
Loader	Dependency, Provider, Config Injection	Runtime Trust
Filesystem	Symlink, Junction, Reparse, Unsafe Parent	Object Identity
Lifecycle	Stale Lease, Crash, Early Shutdown	Lifecycle Controller

ODRE PQC no reemplaza autorización de negocio, corrección de vulnerabilidades de la aplicación, Host Hardening, DDoS, seguridad física, política de retención ni integridad completa del Client Device. Protege el límite de entrada de una solicitud protegida y la confianza del Runtime del que depende.

9. Arquitectura de Despliegue

Cliente con PQC
→ Nginx o Load Balancer
→ FastAPI + ODRE PQC Boundary
→ Protected Handler y Application Data
↳ Native Runtime de plataforma

Nginx o Load Balancer conserva Routing y Transport. ODRE PQC opera dentro de FastAPI antes del Protected Handler. Un diseño Multi-instance considera Session Affinity o Shared State, Sequence Consistency, Runtime Identity por instancia, Worker Leases, Rolling Restart, Retry-induced Replay y separación de Health Checks públicos y protegidos.

Antes del despliegue se verifican OS/Architecture admitidos, Version y Hash del Wheel/Runtime, estado limpio de [doctor](#), resultados PQC reales de [verify](#), política de rutas, ausencia de Automatic Downgrade, procedimiento de Restart/Recovery y Logs/Status sin Secrets.

10. Identidad del Producto

Elemento	Valor
Producto	ODRE PQC
Versión	0.2.9
Official Wheel	odre_pqc-0.2.9-py3-none-any.whl
Wheel SHA-256	A8AFA10EE0AFACEF1C0FAF55FF07B96C722920E5AEE8692F6F026E804F028697
Windows Runtime SHA-256	826303E72A76B10041D26113F465ADD8960825C71F50722D6792174E73F9D12C
Ubuntu Runtime SHA-256	EFAD16698203371D07F25B429F6731B9C0954EEBF5B58A0024789EF0611A78D1
OpenSSL Runtime	3.5.8
PQC	ML-KEM-768 / ML-DSA-65
Protocol	odre-pqc/3
Official OS	Windows Server 2022 AMD64 / Ubuntu 24.04 LTS AMD64

11. Conclusión

ODRE PQC define las condiciones bajo las cuales puede ejecutarse un Application Handler. Runtime Identity, Handshake, Session, Strict Sequence, Replay Defense, Request/Response Binding y Lifecycle se aplican como un contrato único. Una solicitud que no demuestre esas condiciones termina antes del Handler.

El flujo externo permanece [init](#), [doctor](#), [verify](#), [install\(app\)](#) y [status](#), mientras Native Trust por plataforma y estado criptográfico quedan aislados internamente. El valor del producto no es otro nombre de algoritmo, sino una Application-Embedded Post-Quantum Security Boundary cuyas condiciones Allow/Block pueden explicarse, operarse y verificarse.